

July 16, 2026

```
[1]: # Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

from decimal import Decimal, ROUND_CEILING, ROUND_HALF_UP, getcontext #
    ↳ Besonders für sig. Runden
import math

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
import scipy.constants as cc
from scipy.stats import norm
from scipy.optimize import curve_fit
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.integrate import quad
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelmin, argrelmax
from scipy.special import factorial
import scipy.integrate as integrate
from scipy.special import gamma

%matplotlib widget
import matplotlib.pyplot as plt
from matplotlib import colormaps
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path
from pathlib import Path
```

```

import pandas as pd # Auch wichtig für den Latex Export
from pytexit import py2tex
import csv
import re
from typing import List

# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars

# Display und Output
from IPython.display import display, Math, Latex, HTML

import textwrap

# Grafiken in EU-Größen ausgeben
def figsize_cm(width_cm, height_cm):
    return (2*width_cm / 2.54, 2*height_cm / 2.54)
def comma_to_float(valstr):
    return float(valstr.replace(',', '.'))

g=9.80984
Delta_g=0.00002

```

```

[2]: currentversuch = "245"
base = Path.cwd()
Messwerte_path = Path(base.parent.parent.parent/"Messwerte"/currentversuch)
    ↪ # initializing paths
Ausgabe_path = Path(base/"Diagramme")
print(Messwerte_path)
print(Ausgabe_path)

```

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Messwerte\245

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Auswertungen2.2\Code\245\Diagramme

Delete UTF8 False encoding

```

[3]: # def deletevocals(text):
#     text=textwrap.dedent(text)
#     patterna = r'["][a]'
#     text=re.sub(patterna,"ä",text)
#     patternu=r'["][u]'
#     text=re.sub(patternu,"ü",text)
#     patterno=r'["][o]'
#     text=re.sub(patterno,"ö",text)
#     return text

```

```

# Regexpwissen: [0-9], [A-Z] findet egal welchen Zahl/Buchstaben; [^0-9] findet
↳ erstes Zeichen, das keine Zahl ist;

def deletevocal(text):
    # Einrückungen entfernen (gemeinsame Tabs des gesamten Textes)
    text = textwrap.dedent(text)
    # entfernt Bindestriche: - findet Bindestrich, \n Zeilenumbruch direkt
↳ dahinter, \s Whitespaces *beliebig viele
    text = re.sub(r'-\n\s*', "", text)
    # entfernt Zeilenumbrüche
    text = text.replace("\n", " ")

    # Eine Funktion, die vom re.sub aufgerufen wird, wenn ein Treffer erzielt
↳ wird
    def convert(match):
        # match.group(1) ist der Buchstabe nach dem Pünktchen (z.B. 'a' oder
↳ 'A')
        buchstabe = match.group(1)
        # Dictionary für die echten Umlaute
        umlaute = {'a': 'ä', 'u': 'ü', 'o': 'ö', 'A': 'Ä', 'U': 'Ü', 'O': 'Ö'}
        # Gib den Umlaut zurück. Falls das Zeichen nicht im Dict ist, lass es
↳ wie es ist
        return umlaute.get(buchstabe, buchstabe)
    # Das Pattern sucht nach ` gefolgt von einem beliebigen Buchstaben aus der
↳ Auswahl, [X] findet genau ein Zeichen aus Menge X
    return re.sub(r'`([a-zA-Z])', convert, text)

```

```

[4]: print(deletevocal("""
"""))

```

Runden signifikanter Stellen

```

[5]: def round_to_sigs(val, errVal=None, einheiten=None):
    """
    Funktion zur Rundung eines Fehlers und die Anpassung des Messwertes daran.
↳ Diese Funktion wurde etwas umständlicher
    geschrieben, da python mit Floats und Runden schnell in Probleme rennt.
↳ Daher musste hier mit dezimal gearbeitet werden.
    Zudem sollten besonders kleine und große Messwerte in der
↳ Dezimalschreibweise geschrieben werden, damit diese auch
    für Protokolle geeignet sind.

    Parameter

```

```

-----
**val** : float
    Messwert

**errVal** : float
    Ungenauigkeit des Messwertes

Return
-----

**value_rounded** : str
    Gerundeter Messwert

**error_rounded** : str
    Gerundete Ungenauigkeit des Messwertes

**res** : str
    "Messwert \pm Fehler"
"""
einheiten_str = einheiten if einheiten is not None else ""
if errVal is not None:
    # Daten zu Dezimal wechseln, da Floats probleme machen
    val = Decimal(str(val))
    errVal = Decimal(str(errVal))

    exp = int(math.floor(math.log10(float(errVal)))) # Die erste
    ↳signifikante Stellenposition wird anhand des errVal bestimmt

    if round(float(errVal / (Decimal(10) ** exp))) < 3: # wenn
    ↳1,2,3 erste signifikante Stelle, dann kommt eine zweite
    ↳Nachkommastellenposition hinzu
        exp -= 1

    scale = Decimal(10) ** exp

    val_round = (val / scale).quantize(Decimal('1'),
    ↳rounding=ROUND_HALF_UP) * scale # mit scale wird das Komma so
    ↳verschoben, dass alle signifikanten Stellen vor dem Komma stehen, und dann
    ↳alle Nachkommastellen weggerundet werden
    err_round = (errVal / scale).quantize(Decimal('1'),
    ↳rounding=ROUND_CEILING) * scale

    num = f"\num{{{val_round}}({err_round})}"
    qty = f"\qty{{{val_round}}({err_round})}{{{einheiten_str}}}" if
    ↳einheiten else num

    return float(val_round), float(err_round), num, qty
else:

```

```

    val = Decimal(str(val))
    exp = int(math.floor(math.log10(float(val))))
    if round(float(val / (Decimal(10) ** exp))) < 3:           # wenn 1,2,3
↳erste signifikante Stelle, dann kommt eine zweite Nachkommastellenposition
↳hinzu
        exp -= 1
    scale = Decimal(10) ** exp
    val_round = (val / scale).quantize(Decimal('1'),
↳rounding=ROUND_HALF_UP) * scale
    num = f"\\num{{{val_round}}}"
    qty = f"\\qty{{{val_round}}}{einheiten_str}" if einheiten else num
    return float(val_round), None, num, qty

v_round = np.vectorize(round_to_sigs, otypes=[float, float, object, object],
↳excluded=['einheiten'])

    # wissenschaftliche Notation erstmal vernachlässigen (da häufig Einheiten
↳gewollt sind, die 10^e Präfixe beinhalten)

```

Gauss'sche Fehlerfortpflanzung

```

[6]: def gff(function, errPronePar, latex=True):
    """
    Kann die Fehlerformel einer gegebenen Gleichung bestimmen.

    Parameters
    -----
    **func** : sympy function
        Funktion dessen Fehler bestimmt werden soll.

    **errPronePar** : Array
        Liste (Array) aller fehlerbehafteten Größen der Gleichung con sp.Symbols
        Diese Werte werden als x_sym, y_sym, z_sym etc. bezeichnet und sind
↳ungleich den Werten für x, y, z.
        Für die Werte wird daher die Bezeichnung x_val, y_val, z_val etc.
↳genutzt und für deren Fehler err_x, err_y, err_z etc.

    Return
    -----
    **absolut_err** : sympy function
        Gibt die Fehlergleichung des absoluten Fehlers wieder.

    **relativ_err** : sympy function
        Gibt die Fehlergleichung des relativen Fehlers wieder.

    **errProneParameters** : array
        Liste aller Fehlerbehafteten Größen

```

```

"""

error = 0
errProneParameters = []

function = sp.simplify(function)
variables = errPronePar.split(",")
variables = sp.symbols(variables)

for variable in variables:
    Delta = sp.Symbol(f"Delta_{variable}")
    partial = sp.diff(function, variable) # Die Funktion
    ↪ wird nach der fehlerbehafteten Variable abgeleitet
    term = sp.UnevaluatedExpr(partial*Delta)**2
    error = error + ((term)) # Fehler werden
    ↪ quadratisch aufsummiert
    errProneParameters.append((variable, Delta))

absolute_err = sp.simplify(sp.sqrt(error), rational = True)
relative_err = sp.simplify(sp.sqrt(error/function**2), rational = True)

if latex:
    # Prints out the Latex Code for the Functions
    print("Funktion:")
    display(Math(sp.latex(function, long_frac_ratio=2)))
    print(sp.latex(function))
    print("Absoluter Fehler:")
    display(Math(sp.latex(absolute_err, long_frac_ratio=2)))
    print(sp.latex(absolute_err))
    print("Relativer Fehler:")
    display(Math(sp.latex(relative_err, long_frac_ratio=2)))
    print(sp.latex(relative_err))
    return function, absolute_err, relative_err, errProneParameters

```

Sigma-Abweichungen

```

[7]: def sigma_abweichung(p1, err_p1, p2, err_p2):
    """
    Funktion zum berechnen der Sigma-Abweichung von zwei Messwerten, oder einem
    ↪ Messwert und einem Literaturwert.
    """
    if err_p1 == 0 and err_p2 == 0:
        raise ValueError("Für die Sigma-Abweichung muss mindestens ein Wert
        ↪ fehlerbehaftet sein!")
    else:
        abweichung = Decimal(str(abs(p1 - p2)/(np.sqrt(err_p1 ** 2 + err_p2 ** 2))))

```

```

if abweichung == 0:
    return 0.0, "\\num{0}"

exp = int(math.floor(math.log10(float(abweichung))))
if round(float(abweichung / (Decimal(10) ** exp))) < 3:           # wenn
    ↪ 1,2,3 erste signifikante Stelle, dann kommt eine zweite
    ↪ Nachkommastellenposition hinzu
    exp -= 1
scale = Decimal(10) ** exp
abweichung_round = (abweichung / scale).quantize(Decimal('1'),
    ↪ rounding=ROUND_CEILING) * scale
res = f"\\num{{{abweichung_round}}}"

return float(abweichung_round), res

```

```

[8]: # #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2
#     elif val1>val2:
#         return name1+">"+name2
#     else:
#         return name1+"="+name2

#Erstellung von Vergleichstabellen in Latex
def compare_table_array(nam1, nam2, einheiten, val1_array, err1_array,
    ↪ val2_array, err2_array):
    # 1. Tabellenkopf (Hier werden die Strings einmalig eingesetzt)
    output_header = textwrap.dedent(f"""
        \\begin{{table}}[htbp]
        \\centering
        \\caption{{Vergleich von ${nam1}$ und ${nam2}$}}
        \\begin{{tabularx}}{{0.8\\textwidth}}{{ZZZZZ}}
        \\toprule
        Messreihe & {nam1}[\\unit{{{einheiten}}}] &
    ↪ {nam2}[\\unit{{{einheiten}}}] & \\text{{abs.Abw.}}[\\unit{{{einheiten}}}] &
    ↪ \\text{{proz.Abw.}}[\\unit{{\\percent}}] & S[\\sigma] \\midrule
        """).strip() # strip macht whitespaces am anfang und ende des strings weg

    table_rows = []

    # 2. Der Body (Schleife läuft über die Werte-Arrays)
    for val1, err1, val2, err2 in zip(val1_array, err1_array, val2_array,
    ↪ err2_array):

        VAL1=round_to_sigs(val1,err1,r'[2]
        if err2!=0:

```

```

        VAL2=round_to_sigs(val2,err2,r'')[2]
    else:
        VAL2=val2

    abs=np.abs(val1-val2)
    abs_delta=np.sqrt(err1**2+err2**2)
    if abs_delta!=0:
        SIGMA=sigma_abweichung(val1,err1,val2,err2)[1]
    else:
        SIGMA=0.0

    rel=abs/np.abs(val1)*100
    if abs != 0:
        rel_delta = rel * np.sqrt((abs_delta / abs)**2 + (err1 / val1)**2)
        ABS = round_to_sigs(abs, abs_delta, r'')[2]
        REL = round_to_sigs(rel, rel_delta, r'')[2]
    else:
        rel_delta = 0
        ABS = "0"
        REL = "0"

    # Eine saubere Zeile nur mit den Werten für LaTeX
    row = f"          & {VAL1} & {VAL2} & {ABS} & {REL} & {SIGMA} \\\\"
    table_rows.append(row)

body_str = "\\n".join(table_rows)

# 3. Tabellenfuß
output_footer = textwrap.dedent(f"""
    \\bottomrule
    \\end{{tabularx}}
    \\label{{table:Vergleich_{nam1}}}
    \\end{{table}}
""").strip()

# Alles zusammensetzen
final_output = f"{output_header}\\n{body_str}\\n{output_footer}"

print(final_output)

```

```

[9]: # #Erstellung von Vergleichstabellen in Latex
# def compare_table_withliterature(nam1,nam2,einheiten,val1,err1,val2):
#     VAL1=round_to_sigs(val1,err1,r'')[2]
#     abs=np.abs(val1-val2)
#     abs_delta=np.sqrt(err1**2)
#     if abs_delta!=0:
#         SIGMA=sigma_abweichung(val1,err1,val2,0)[1]

```



```

else:
#         SIGMA=0.0
#         ABS=round_to_sigs(abs,abs_delta,r'')[2]
#         rel=abs/np.abs(val2)*100
#         rel_delta=rel*np.sqrt((abs_delta/abs)**2+(err1/val1)**2) if abs != 0 else 0
#         REL=round_to_sigs(rel,rel_delta,r'')[2]
#         output = textwrap.dedent(f"""
#             \begin{table}[htb]
#             \centering
#             \caption{}
#             \begin{tabularx}{0.8\textwidth}{ZZZZZ}
#             \toprule
#             \hline
#             \begin{array}{ccccc}
#             \{nam1\} & \text{\texttt{\textbackslash unit\{\{einheiten\}\}}} & \& \{nam2\} & \text{\texttt{\textbackslash unit\{\{einheiten\}\}}} & \& \text{\texttt{\textbackslash text\{abs.Abw.}} \\
#             \& \text{\texttt{\textbackslash unit\{\{einheiten\}\}}} & \& \text{\texttt{\textbackslash text\{proz.Abw.}} \\
#             \& \text{\texttt{\textbackslash unit\{\{percent\}\}}} & \& \text{\texttt{\textbackslash sigma}} & \text{\texttt{\textbackslash \midrule}} \\
#             \{VAL1\} & \& \text{\texttt{\textbackslash num\{\{val2\}\}}} & \& \{ABS\} & \& \{REL\} & \& \{SIGMA\} \\
#             \text{\texttt{\textbackslash bottomrule}} \\
#             \text{\texttt{\textbackslash end\{\{tabularx\}\}}} \\
#             \text{\texttt{\textbackslash label\{\{table:Vergleich_{nam1}\}\}}} \\
#             \text{\texttt{\textbackslash end\{\{table\}\}}}
#             """)
#         print(output)

# #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2
#     elif val1>val2:
#         return name1+">"+name2
#     else:
#         return name1+"="+name2

#Erstellung von Vergleichstabellen in Latex
import textwrap
def compare_table(nam1,nam2,einheiten,val1,err1,val2,err2):
    VAL1=round_to_sigs(val1,err1,r'')[2]
    if err2!=0:
        VAL2=round_to_sigs(val2,err2,r'')[2]
    else:
        VAL2=val2
    abs=np.abs(val1-val2)
    abs_delta=np.sqrt(err1**2+err2**2)
    if abs_delta!=0:

```

```

        SIGMA=sigma_abweichung(val1,err1,val2,err2)[1]
    else:
        SIGMA=0.0

    rel=abs/np.abs(val1)*100
    if abs != 0:
        rel_delta = rel * np.sqrt((abs_delta / abs)**2 + (err1 / val1)**2)
        ABS = round_to_sigs(abs, abs_delta, r')[2]
        REL = round_to_sigs(rel, rel_delta, r')[2]
    else:
        rel_delta = 0
        ABS = "0"
        REL = "0"

    output = textwrap.dedent(f"""
        \\begin{{table}}[htbp]
        \\centering
        \\caption{{Vergleich von ${nam1}$ und ${nam2}$}}
        \\begin{{tabularx}}{{\\textwidth}}{{ZZZx}}
        \\toprule
        \\
        ↪{{nam1}}[\\unit{{einheiten}}]&{{nam2}}[\\unit{{einheiten}}]&\\text{{abs.Abw.
        ↪}}[\\unit{{einheiten}}]&\\text{{proz.Abw.
        ↪}}[\\unit{{percent}}]&S[\\sigma]\\\\\\midrule
        {VAL1}&{VAL2}&{ABS}&{REL}&{SIGMA}\\\\\\
        \\bottomrule
        \\end{{tabularx}}
        \\label{{table:Vergleich_{{nam1}}}}
        \\end{{table}}
    """)
    print(output)

```

```
[39]: print(np.array([0.64,0.93,1.95,2.64,3.5])*np.sqrt(2))
```

```
[0.90509668 1.31521861 2.75771645 3.7335238 4.94974747]
```

```
[41]: U_f=np.array([0.64,0.93,1.95,2.64,3.5])*np.sqrt(2)
# U=np.array([2,2.68,5.54,7.64,9.60])/2
Delta_U_f=0.02
f=np.array([4.7,5.9,9.1,12.1,15.1])
Delta_f=0.1

def linear(x,m,c):
    return m*x+c

plt.close('all')
fig, (ax1,ax2) = plt.subplots(1, 2, figsize=(figsize_cm(14,6)))

```

```

fig.suptitle(r'Aufgabe 1:  $U_{\text{max}}(f, I_H)$ ')

ax1.errorbar(f, U_f, yerr=Delta_U_f, xerr=Delta_f, fmt='.', ecolor='black', capsize=
    ↳ 3, elinewidth= 0.5, label='Messwerte')
popt, pcov=curve_fit(linear, f, U_f, sigma=Delta_U_f, absolute_sigma=True)
x=np.linspace(np.min(f), np.max(f), 10)
ax1.plot(x, linear(x, *popt), label='Linear Fit')
ax1.set_title(r' $U(f, I_H=\text{const})$ : variierte Drehfrequenz, konstantes ext.
    ↳ Feld')
ax1.set_ylabel(r'Maximalspannung  $U_{\text{max}}$  [V]')
ax1.set_xlabel(r'Drehfrequenz  $f$  [Hz]')

I=np.arange(0.5, 5, 0.5)
Delta_I=I*0.00075+0.02
U_I=np.array([328, 599, 840, 1140, 1400, 1660, 1890, 2170, 2410])*np.sqrt(2)
Delta_U_I=20

ax2.errorbar(I, U_I, xerr=Delta_I, yerr=Delta_U_I, fmt=".",
    ↳ ", ecolor='black', capsize=3, elinewidth=0.5, label='Messwerte')
poptI, pcovI=curve_fit(linear, I, U_I, sigma=Delta_U_I, absolute_sigma=True)
x=np.linspace(np.min(I), np.max(I), 10)
ax2.plot(x, linear(x, *poptI), label='Linear Fit')
ax2.set_title(r' $U(I_H, f=\text{const})$ : variierte ext.Feld, konstante
    ↳ Drehfrequenz')
ax2.set_ylabel(r' $U_{\text{eff}}$  [mV]')
ax2.set_xlabel(r'Stromstärke Helmholtz  $I_H$  [A]')

ax1.legend()
# ax1.grid()
ax2.legend()
# ax2.grid()

plt.savefig(Ausgabe_path/'1.png', format="png", bbox_inches='tight')
plt.tight_layout()
plt.show()

m, Delta_m, _, latexm=round_to_sigs(popt[0], np.sqrt(pcov[0][0]), r'\volt\per\hertz')
print("links:")
print(f" $m={\text{latexm}}$ ")
print("c="+round_to_sigs(popt[1], np.sqrt(pcov[1][1]), r'\milli\volt')[3])
print("rechts:")
print(r" $m="+round_to_sigs(poptI[1], np.
    ↳ sqrt(pcovI[0][0]), r'\milli\volt\per\ampere')[3])$ ")
print(r" $c="+round_to_sigs(poptI[0], np.sqrt(pcovI[1][1]), r'\milli\volt')[3])$ ")

n=4000
A=41.7*10**(-4)

```

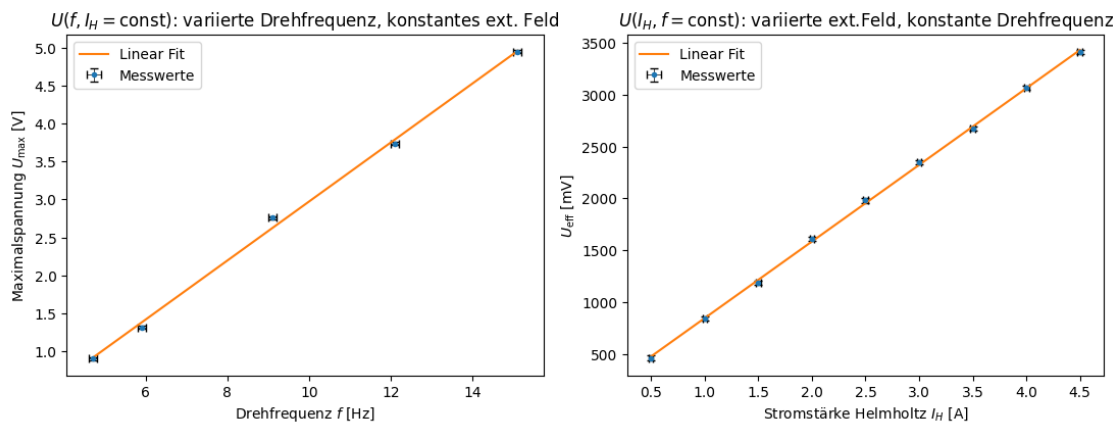
```

B=m/(A*n*2*np.pi)*1e3
Delta_B=np.sqrt(Delta_m**2/(A**2*n**2))/(2*pi)*1e3
B,Delta_B,_,latexB=round_to_sigs(B,Delta_B,r'\milli\tesla')
print(latexB)

I=4
Delta_I=0.0075*I+0.02
B_H=cc.mu_0*8*124*I/(np.sqrt(125)*295*1e-3/2)*1e3
Delta_B_H=B_H*np.sqrt(Delta_I**2/I**2)
B_H,Delta_B_H,_,latexBh=round_to_sigs(B_H,Delta_B_H,r'\milli\tesla')
print(latexBh)
compare_table(r'B',r'B_H',r'\milli\tesla',B,Delta_B,B_H,Delta_B_H)

```

Aufgabe 1: $U_{\max}(f, I_H)$



links:

$m = \text{0.3892(0.0024)} \frac{\text{V}}{\text{Hz}}$

$c = \text{-0.919(0.024)} \text{ mV}$

rechts:

$m = \text{109(6)} \frac{\text{mV}}{\text{A}}$

$c = \text{738(15)} \text{ mV}$

$\text{3.714(0.023)} \text{ mT}$

$\text{3.02(0.04)} \text{ mT}$

$\begin{array}{c} \backslash \text{begin}\{table\} [\text{htbp}] \\ \backslash \text{centering} \\ \backslash \text{caption}\{\text{Vergleich von } B \text{ und } B_H\} \\ \backslash \text{begin}\{tabularx\}\{\text{\textwidth}\}\{\text{ZZZZx}\} \\ \backslash \text{toprule} \\ B [\text{mT}] \& B_H [\text{mT}] \& \text{abs. Abw.} [\text{mT}] \\ \& \text{proz. Abw.} [\text{\%}] \& S [\sigma] \\ \& \text{3.714(0.023)} \& \text{3.02(0.04)} \& \text{0.69(0.05)} \& \text{18.7(1.3)} \& \text{16} \\ \backslash \text{bottomrule} \end{array}$

```

\end{tabularx}
\label{table:Vergleich_B}
\end{table}

```

```

[51]: def linear(x,m,a):
        return m*x+a
plt.close('all')
U=np.array([850,789,501,61.6,380,711,856])*np.sqrt(2)
Delta_U=2*np.sqrt(2)
phi=np.radians(np.arange(0,210,30))
Delta_phi=np.radians(1.5)

def abscos(x, a,b):
    return np.abs(a * np.cos(x))+b

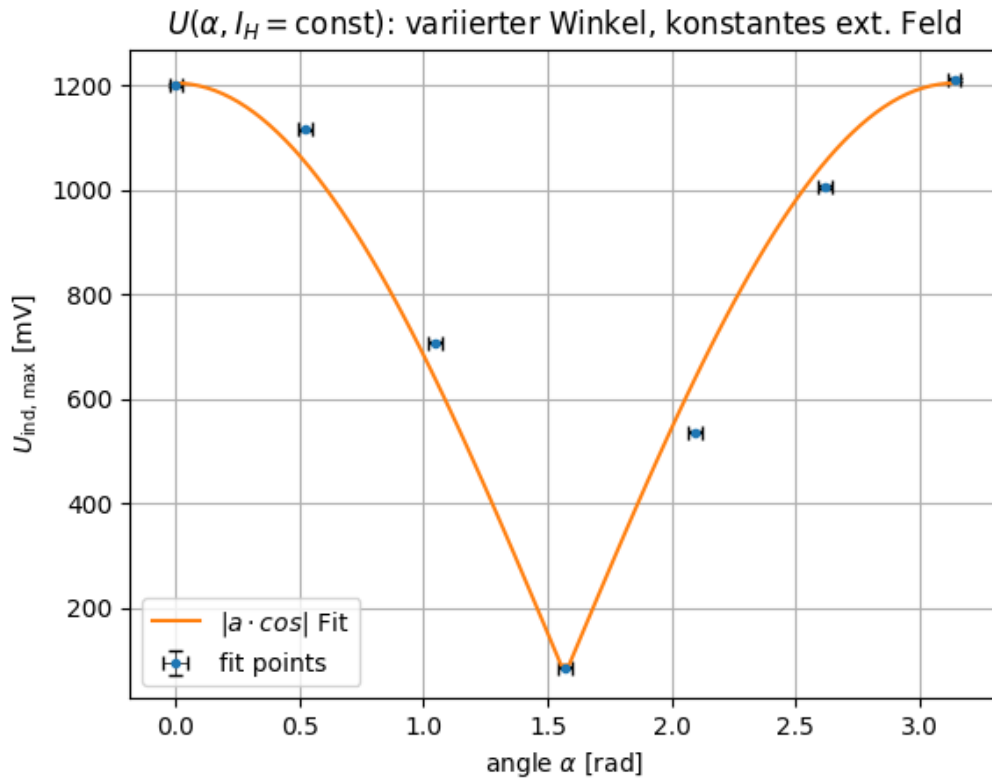
plt.errorbar(phi,U,yerr=Delta_U,xerr=Delta_phi,fmt='.
↳',ecolor='black',capsize=3,elinewidth=0.5,label='fit points')
popt, pcov=curve_fit(abscos,phi,U,sigma=Delta_U,absolute_sigma=True)
x=np.linspace(np.min(phi),np.max(phi), 100)
plt.plot(x, abscos(x,*popt),label=r'$|a\cdot \cos|$ Fit')
print(round_to_sigs(popt[1],np.sqrt(pcov[1][1]),r'\milli\volt')[3])
plt.xlabel(r'angle $\alpha$ [rad]')
plt.ylabel(r'$U_{\text{ind, max}}$ [mV]')
plt.title(r'$U(\alpha, I_H=\text{const})$: variierter Winkel, konstantes ext.
↳Feld')
plt.legend()
plt.grid()
plt.show()
plt.savefig(Ausgabe_path/"task2_angle.png",format="png")

```

```

\qty{73.1(2.4)}{\milli\volt}

```



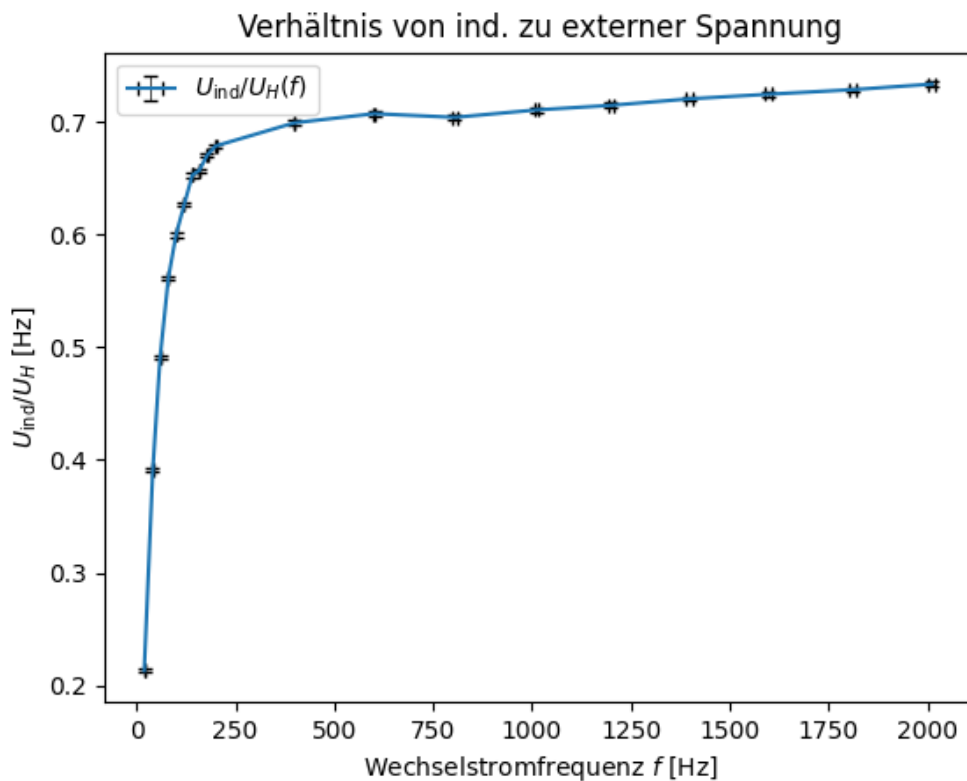
```
[42]: # Messwerte
t4frequency = np.array([19.3,41.0,59.5,80.5,99.9,120.4,141.0,158.5,178.
    ↪5,200,398.5,601,805.0,1010.0,1200,1397.0,1597.0,1812.0,2012.0])
t4frequency_delta = np.array([0.3,0.3,0.3,0.3,0.3,0.3,1.5,1,2.0,2.0,2.0,2.
    ↪0,5,5,5,10,10,10,10])

t4max_voltage_ind = np.array([0.245,0.473,0.604,0.691,0.744,0.777,0.790,0.808,0.
    ↪818,0.828,0.860,0.863,0.859,0.867,0.872,0.879,0.884,0.889,0.895])
t4max_voltage_ind_delta = 0.002
t4max_voltage_coil = np.array([1.15,1.21,1.23,1.23,1.24,1.24,1.21,1.23,1.22,1.
    ↪22,1.23,1.22,1.22,1.22,1.22,1.22,1.22,1.22,1.22])
t4max_voltage_coil_delta = 0.002
t4max_current_coil = np.array([152.5,135.0,117.0,99.2,86.2,74.4,64.8,58.9,52.
    ↪9,48.0,25.1,16.8,12.4,9.94,8.39,7.21,6.29,5.56,5.00])*(10**(-3))
t4max_current_coil_delta = t4max_current_coil*0.009+0.0002
t4max_voltage_ratio = t4max_voltage_ind/t4max_voltage_coil
t4max_voltage_ratio_delta = t4max_voltage_ratio*np.
    ↪sqrt((t4max_voltage_coil_delta/
    ↪t4max_voltage_coil)**2+(t4max_voltage_ind_delta/t4max_voltage_ind)**2)
plt.close('all')
```

```

plt.xlabel(r'Wechselstromfrequenz $f$ [Hz]')
plt.ylabel(r'$U_{\text{ind}}/U_H$ [Hz]')
plt.title(r'Verhältnis von ind. zu externer Spannung')
plt.errorbar(t4frequency, t4max_voltage_ratio, xerr = np.
    ↳full_like(t4frequency, t4frequency_delta) , yerr=np.
    ↳full_like(t4max_voltage_ratio, t4max_voltage_ratio_delta), ecolor='black', capsize=3, elinewidtht.
    ↳5, fmt='-', label=r'$U_{\text{ind}}/U_H(f)$')
plt.legend()
plt.show()
plt.savefig(Ausgabe_path/"task2_ratio.png", format="png")

```



```

[52]: t4resistance = t4max_voltage_coil/t4max_current_coil
t4resistance_delta = t4resistance*np.sqrt(((t4max_voltage_coil_delta)/
    ↳t4max_voltage_coil)**2+ ((t4max_current_coil_delta)/t4max_current_coil)**2)
xfit = np.linspace(0, np.max(t4frequency), 100)
def impedance(f,R,L):
    return np.sqrt(R**2+(2*pi*f*L)**2)
plt.close('all')
plt.xlabel(r'Wechselstromfrequenz $f$ [Hz]')
plt.ylabel(r'Gesamtimpedanz $|Z_H|$ $[\Omega]$')

```

```

plt.errorbar(t4frequency, t4resistance, xerr = t4frequency_delta,
    ↳yerr=t4resistance_delta, fmt = '.',ecolor='black',capsize=3,elinewidth=0.
    ↳5,label='Messpunkte')

popt, pcov = curve_fit(impedance,t4frequency, t4resistance,
    ↳sigma=t4resistance_delta,absolute_sigma=True)
R,Delta_R,_,latexR=round_to_sigs(popt[0],np.sqrt(pcov[0][0]),r'\ohm')
print(latexR)
L,Delta_L,_,latexL=round_to_sigs(popt[1],np.sqrt(pcov[1][1]),r'\henry')
print(latexL)

plt.plot(xfit, impedance(xfit,*popt), label = r'\sqrt{R^2+(2\pi fL)^2}$,
    ↳Fit',linestyle='-',color = 'purple')
plt.legend()
plt.savefig(Ausgabe_path/"task2_R.png",format="png")

chi2_g=np.sum((impedance(t4frequency,*popt)-t4resistance)**2/
    ↳t4resistance_delta**2)
dof_g=len(t4frequency)-2 #dof:degrees of freedom, Freiheitsgrad
chi2_red_g=chi2_g/dof_g
print("chi2_g=", round_to_sigs(chi2_g)[0])
print("chi2_red_g=",round_to_sigs(chi2_red_g)[0])

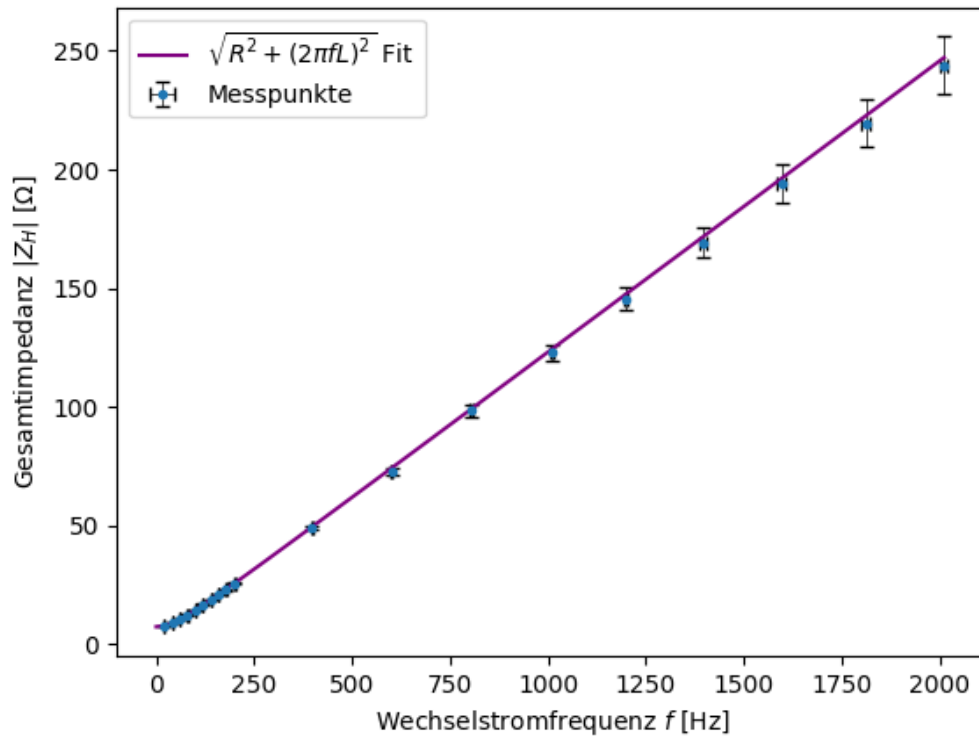
from scipy.stats import chi2
#Gauss:
prob_g=round(1-chi2.cdf(chi2_g,dof_g),2)*100
#Poisson:
print("Wahrscheinlichkeit Gauss=", prob_g,"%")

```

```

\qty{-7.32(0.07)}{\ohm}
\qty{0.01954(0.00011)}{\henry}
chi2_g= 11.0
chi2_red_g= 0.7
Wahrscheinlichkeit Gauss= 84.0 %

```

```
[44]: NFlach=4000
AFlach=41.7*1e-4
NperCoilHelm=124
diameter=0.295
print(58*1e-3*np.sqrt(2)*2)

U_sp1 = 58*1e-3*np.sqrt(2)*2
U_sp1_delta = 3e-3*2*np.sqrt(2)
f1 = 15.1
f1_delta = 0.2
U_sp2 = 59.2e-3
U_sp2_delta = 5e-3
f2 = 15.1
f2_delta = 0.2
I_komp = 70.6e-3
I_komp_delta = 0.5e-3

Bges = (U_sp1/2)/(NFlach*AFlach*2*np.pi*f1) *1e6#B_gesamt
deltaBges = Bges * np.sqrt((U_sp1_delta/U_sp1)**2+(f1_delta/f1)**2)
#zugehörigeFehler
```

```

Bges,deltaBges,_,latexBges=round_to_sigs(Bges,deltaBges,r'\mikro\tesla')
print('B')
print(latexBges)

Bhor = (U_sp2/2)/(NFlach*AFlach*2*np.pi*f2) *1e6 #horizontaler Anteil
deltaBhor = Bhor * np.sqrt((U_sp2_delta/U_sp2)**2+(f2_delta/f2)**2)
B_H,Delta_B_H,_,latexBhor=round_to_sigs(Bhor,deltaBhor,r'\mikro\tesla')
print('Bhor')
print(latexBhor)

Bvert = 8/(np.sqrt(125)) * cc.mu_0 * NperCoilHelm * I_komp / (diameter/2)
↳*1e6#vertikaler Anteil
deltaBvert = Bvert * (I_komp_delta/I_komp)
B_V,Delta_B_V,_,latexBvert=round_to_sigs(Bvert,deltaBvert,r'\mikro\tesla')
print('Bver')
print(latexBvert)

IncAng = np.arctan(Bvert/Bhor) #Inklinationswinkel
IncAng = np.degrees(IncAng)
deltaIncAng = np.sqrt(((Bhor*deltaBvert)/(Bhor**2 + Bvert**2))**2 +
↳((Bvert*deltaBhor)/(Bhor**2 + Bvert**2))**2)
deltaIncAng = np.degrees(deltaIncAng)
IncAng,deltaIncAng,_,latexincang=round_to_sigs(IncAng,deltaIncAng,r'\degree')
print(latexincang)

B_ges=np.sqrt(Bvert**2+Bhor**2)
Delta_B_ges= np.sqrt((B_H**2*Delta_B_H**2 + B_V**2*Delta_B_V**2)/(B_H**2 +
↳B_V**2))
B_ges,Delta_B_ges,_,latexBges=round_to_sigs(B_ges,Delta_B_ges,r'\mirco\tesla')
print(latexBges)

```

0.16404877323527906

B

\qty{52(3)}{\mikro\tesla}

Bhor

\qty{18.7(1.6)}{\mikro\tesla}

Bver

\qty{53.4(0.4)}{\mikro\tesla}

\qty{70.7(1.6)}{\degree}

\qty{56.6(0.7)}{\mirco\tesla}

[19]: `gff("sqrt(B_H**2+B_V**2)","B_H,B_V")`

Funktion:

$$\sqrt{B_H^2 + B_V^2}$$

\sqrt{B_{H}^2 + B_{V}^2}

Absoluter Fehler:

$$\sqrt{\frac{1}{B_H^2 + B_V^2} (B_H^2 \Delta_{BH}^2 + B_V^2 \Delta_{BV}^2)}$$

$$\sqrt{\frac{B_H^2 \Delta_{BH}^2 + B_V^2 \Delta_{BV}^2}{B_H^2 + B_V^2}}$$

Relativer Fehler:

$$\sqrt{\frac{1}{(B_H^2 + B_V^2)^2} (B_H^2 \Delta_{BH}^2 + B_V^2 \Delta_{BV}^2)}$$

$$\sqrt{\frac{B_H^2 \Delta_{BH}^2 + B_V^2 \Delta_{BV}^2}{(B_H^2 + B_V^2)^2}}$$

```
[19]: (sqrt(B_H**2 + B_V**2),
      sqrt((B_H**2*Delta_B_H**2 + B_V**2*Delta_B_V**2)/(B_H**2 + B_V**2)),
      sqrt((B_H**2*Delta_B_H**2 + B_V**2*Delta_B_V**2)/(B_H**2 + B_V**2)**2),
      [(B_H, Delta_B_H), (B_V, Delta_B_V)])
```

```
[29]: compare_table(r'B_\text{exp}',r'B_\text{Lit}',r'\micro\tesla',Bges,deltaBges,49,0)
      compare_table(r'B_\text{Pythagoras}',r'B',r'\micro\tesla',Bges,deltaBges,B_ges,Delta_B_ges)
      compare_table(r'\alpha_\text{exp}',r'\alpha_\text{Lit}',r'\degree',70.7,1.
      ↪6,66,0)
```

```
\begin{table}[htbp]
\centering
\caption{Vergleich von $B_\text{exp}$ und $B_\text{Lit}$}
\begin{tabularx}{\textwidth}{ZZZZx}
\toprule
B_\text{exp}[\unit{\micro\tesla}]&B_\text{Lit}[\unit{\micro\tesla}]&\text{abs. Abw.}[\unit{\micro\tesla}]&\text{proz. Abw.}[\unit{\percent}]&S[\sigma]\\\midrule
\num{52(3)}&\num{3(3)}&\num{6(6)}&\num{1.0}\\\bottomrule
\end{tabularx}
\label{table:Vergleich_B_\text{exp}}
\end{table}
```

```
\begin{table}[htbp]
\centering
\caption{Vergleich von $B_\text{Pythagoras}$ und $B$}
\begin{tabularx}{\textwidth}{ZZZZx}
\toprule
B_\text{Pythagoras}[\unit{\micro\tesla}]&B[\unit{\micro\tesla}]&\text{abs. Abw.}[\unit{\micro\tesla}]&\text{proz. Abw.}[\unit{\percent}]&S[\sigma]\\\midrule
\end{tabularx}
\end{table}
```

```

\num{52(3)}&\num{56.6(0.7)}&\num{5(4)}&\num{9(6)}&\num{1.5}\\
\bottomrule
\end{tabularx}
\label{table:Vergleich_B_\text{Pythagoras}}
\end{table}

\begin{table}[htbp]
\centering
\caption{Vergleich von $\alpha_\text{exp}$ und $\alpha_\text{Lit}$}
\begin{tabularx}{\textwidth}{ZZZx}
\toprule
\alpha_\text{exp} [\unit{\degree}] & \alpha_\text{Lit} [\unit{\degree}] & \text{abs. Abw.} [\unit{\degree}] & \text{proz. Abw.} [\unit{\percent}] & S [\unit{\sigma}] \\
\midrule
\bottomrule
\end{tabularx}
\label{table:Vergleich_\alpha_\text{exp}}
\end{table}

```

[17]: `gff("arctan(B_V/B_H)", "B_V,B_H")`

Funktion:

$$\arctan\left(\frac{B_V}{B_H}\right)$$

`\arctan{\left(\frac{B_{V}}{B_{H}}\right)}`

Absoluter Fehler:

$$\sqrt{\frac{1}{B_H^4} (B_H^2 \Delta_{BV}^2 + B_V^2 \Delta_{BH}^2) \left. \frac{d}{d\xi_1} \arctan(\xi_1) \right|_{\xi_1 = \frac{B_V}{B_H}}^2}$$

`\sqrt{\frac{\left(B_H^2 \Delta_{BV}^2 + B_V^2 \Delta_{BH}^2\right)}{B_H^4} \left. \frac{d}{d\xi_1} \arctan\left(\xi_1\right) \right|_{\xi_1 = \frac{B_V}{B_H}}}`

Relativer Fehler:

$$\sqrt{\frac{1}{B_H^4 \arctan^2\left(\frac{B_V}{B_H}\right)} (B_H^2 \Delta_{BV}^2 + B_V^2 \Delta_{BH}^2) \left. \frac{d}{d\xi_1} \arctan(\xi_1) \right|_{\xi_1 = \frac{B_V}{B_H}}^2}$$

`\sqrt{\frac{\left(B_H^2 \Delta_{BV}^2 + B_V^2 \Delta_{BH}^2\right)}{B_H^4 \arctan^2\left(\frac{B_V}{B_H}\right)} \left. \frac{d}{d\xi_1} \arctan\left(\xi_1\right) \right|_{\xi_1 = \frac{B_V}{B_H}}}`

```

[17]: (arctan(B_V/B_H),
      sqrt((B_H**2*Delta_B_V**2 + B_V**2*Delta_B_H**2)*Subs(Derivative(arctan(_xi_1),
_xi_1), _xi_1, B_V/B_H)**2/B_H**4),
      sqrt((B_H**2*Delta_B_V**2 + B_V**2*Delta_B_H**2)*Subs(Derivative(arctan(_xi_1),
_xi_1), _xi_1, B_V/B_H)**2/(B_H**4*arctan(B_V/B_H)**2)),
      [(B_V, Delta_B_V), (B_H, Delta_B_H)])

```